TECNOLÓGICO DE MONTERREY

Actividad 2.2
Aprendizaje Supervisado

Dataset utilizado:

Breast Cancer (University Medical Centre, Ljubljana, Yugoslavia)

Integrantes del equipo

Abigail Pérez García | A01736815
Alvaro Alberto Cruz Jiménez | A01737453
Fernando Maggi Llerandi | A01736935
Rodrigo López Guerra | A01737437

Fecha de entrega:
27 de Abril 2026

I. Selección de Datos y Preprocesamiento

El dataset seleccionado para esta actividad es el conjunto de datos de cáncer de mama (Breast Cancer) del Instituto de Oncología de la Universidad de Ljubljana, Yugoslavia. Este dataset contiene 286 instancias clasificadas en dos categorías: no-recurrence-events (201 instancias, 70.3%) y recurrence-events (85 instancias, 29.7%). Los datos están descritos mediante 9 atributos, todos de tipo categórico, lo que los convierte en un caso representativo de preprocesamiento complejo.

1.1 Descripción del dataset

Atributo	Tipo	Valores posibles	Faltantes
age	Ordinal	10-19, 20-29, …, 90-99	0
menopause	Ordinal	lt40, premeno, ge40	0
tumor-size	Ordinal	0-4, 5-9, …, 55-59	0
inv-nodes	Ordinal	0-2, 3-5, …, 36-39	0
node-caps	Nominal binario	yes, no	8
deg-malig	Ordinal numérico	1, 2, 3	0
breast	Nominal binario	left, right	0
breast-quad	Nominal	left-up, left-low, right-up, right-low, central	1
irradiat	Nominal binario	yes, no	0
Class	Objetivo	no-recurrence-events, recurrence-events	0

1.2 Manejo de valores faltantes

Los valores faltantes en el dataset están representados con el símbolo "?". En total se detectaron 9 instancias afectadas: 8 en node-caps y 1 en breast-quad. Dado que todos los atributos son categóricos, se eligió imputar con la moda de cada columna. Este criterio es apropiado porque preserva la distribución original de las categorías sin introducir valores artificiales continuos.

1.3 Codificación de atributos a tipo numérico

Cada atributo recibió un tratamiento diferente según su naturaleza semántica:

- Atributos ordinales (***age, menopause, tumor-size, inv-nodes***): se aplicó codificación ordinal respetando el orden natural inherente en los rangos. Por ejemplo, "20-29" < "30-39" < "40-49", lo que preserva la información de magnitud.

- ***deg-malig***: ya contiene valores enteros (1, 2, 3) con orden natural; se convirtió directamente a tipo entero.
- Atributos binarios (***node-caps, breast, irradiat***): mapeados a 0/1, eliminando la necesidad de columnas adicionales.
- ***breast-quad***: es el único atributo nominal sin orden entre sus cinco categorías. Se aplicó One-Hot Encoding, generando cinco columnas indicadoras (una por categoría). Este método es el más adecuado porque evita introducir artificialmente una relación ordinal entre categorías que no la tienen.
- ***Class***: variable objetivo binaria; no-recurrence-events → 0, recurrence-events → 1.

1.4 Normalización

Los atributos ordinales resultantes (age, menopause, tumor-size, inv-nodes, deg-malig) presentan rangos numéricos distintos después de la codificación. Se aplicó normalización Min-Max para escalarlos al intervalo [0, 1]. Esta transformación es especialmente crítica para kNN, cuyo cálculo de distancias es directamente sensible a la escala: sin normalización, atributos con rangos más amplios dominarían la medida de similitud de forma artificial.

1.5 División en conjuntos de entrenamiento y evaluación

El conjunto de datos se dividió en 70% para entrenamiento (200 instancias) y 30% para evaluación (86 instancias), utilizando el parámetro stratify=y en la función train_test_split de scikit-learn. Este parámetro garantiza que la proporción de clases (~70.3% / ~29.7%) se conserve en ambos subconjuntos, evitando particiones donde una clase quede subrepresentada, lo cual es especialmente importante dado el desbalance del dataset. La gráfica de barras por subconjunto confirma que las proporciones se mantienen correctamente.

II. Investigación Documental: Random Forest

El clasificador investigado es Random Forest, un método de ensamble que construye múltiples árboles de decisión y combina sus predicciones para obtener resultados más robustos y generalizables que un árbol individual.

2.1 ¿Cómo funciona?

Random Forest combina dos principios fundamentales: Bootstrap Aggregating (Bagging) y selección aleatoria de características. Durante el entrenamiento, cada árbol del bosque se construye sobre una muestra distinta obtenida con reemplazo del conjunto de entrenamiento original (bagging). En cada nodo de cada árbol, solo se evalúa un subconjunto aleatorio de atributos (determinado por el parámetro max_features) para decidir la mejor división. Esta doble aleatoriedad decorrelaciona los árboles entre sí, reduciendo la varianza del modelo resultante.

En la fase de predicción, cada árbol del bosque emite un voto de clase y el resultado final es la clase con más votos (votación mayoritaria). Al promediar múltiples predictores con baja correlación, el ensamble logra una exactitud superior a la de cualquier árbol individual, manteniendo una resistencia considerable al sobreajuste.

2.2 Tipos de datos que maneja

Random Forest puede trabajar con datos numéricos y categóricos (aunque la implementación de scikit-learn requiere que los datos estén codificados numéricamente previamente). Tolera bien valores atípicos, no requiere que los datos sigan una distribución particular y es capaz de capturar relaciones no lineales entre atributos.

2.3 Parámetros principales

Parámetro	Descripción	Valor por defecto
n_estimators	Número de árboles en el bosque	100
max_depth	Profundidad máxima de cada árbol	None (sin límite)
max_features	Nº de atributos evaluados por nodo ('sqrt', 'log2', int)	'sqrt'
min_samples_split	Mínimo de muestras para dividir un nodo interno	2
min_samples_leaf	Mínimo de muestras requeridas en una hoja	1
criterion	Función de impureza para evaluar divisiones (gini, entropy)	'gini'
bootstrap	Si se usan muestras bootstrap para entrenar cada árbol	True

2.4 Tipo de modelo que genera

Random Forest genera un modelo de tipo caja negra compuesto por múltiples árboles de decisión. Si bien cada árbol individual es interpretable, el conjunto completo no lo es directamente. Sin embargo, el algoritmo proporciona la métrica feature_importances_, que cuantifica la contribución promedio de cada atributo a las divisiones a lo largo de todos los árboles, ofreciendo una herramienta útil de interpretación indirecta.

2.5 Fuentes consultadas

- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
- Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
- Scikit-learn documentation: sklearn.ensemble.RandomForestClassifier. https://scikit-learn.org/stable/modules/ensemble.html#forest

III. Implementación de Algoritmos

Se implementaron tres clasificadores: kNN de forma manual (sin librerías automáticas para el cálculo de distancias), árbol de decisión con scikit-learn, y Random Forest con scikit-learn.

3.1 kNN — Implementación Propia (sin librerías automáticas)

Distancia de Minkowski

La elección de la Distancia de Minkowski obedece a su naturaleza versátil, permitiendo parametrizar la geometría del espacio mediante el valor p. Esta flexibilidad es vital en el análisis de datos médicos, ya que permite ser robusto ante valores atípicos mediante la distancia Manhattan (p=1) o realizar análisis de proximidad estándar con la distancia Euclidiana (p=2).

Como optimización técnica se aplicó un casteo obligatorio a float y el uso de abs() para garantizar estabilidad y prevenir errores por tipos de datos inconsistentes.

Voto Mayoritario (con o sin ponderación)

Este componente consolida la información de los k vecinos más cercanos para asignar una etiqueta definitiva. En modo ponderado, los vecinos más cercanos tienen mayor influencia (peso = 1/distancia). Se integra una constante de estabilidad (1e-9) para prevenir errores de división por cero.

Predicción de instancias

El núcleo operativo del kNN recorre todo el conjunto de entrenamiento calculando la distancia de Minkowski entre la nueva instancia y cada registro, ordena los resultados y selecciona los k vecinos más cercanos para determinar la clase predicha por voto.

3.2 Árbol de Decisión (scikit-learn)

Se utilizó la clase DecisionTreeClassifier de scikit-learn. El árbol aprende reglas de decisión jerárquicas a partir de los datos de entrenamiento, seleccionando en cada nodo el atributo y umbral que minimizan la impureza (Gini o entropía) de los subconjuntos resultantes. El modelo entrenado puede visualizarse tanto en forma textual (export_text) como gráfica (plot_tree).

3.3 Random Forest (scikit-learn)

Se utilizó la clase RandomForestClassifier de scikit-learn, que automatiza la construcción del ensamble de árboles. Internamente gestiona el proceso de bootstrap, la selección aleatoria de atributos y la agregación de votos.

IV. Experimentación y Evaluación

Para cada algoritmo se realizaron ocho experimentos variando los parámetros más relevantes. Se obtuvo la exactitud tanto en el conjunto de entrenamiento como en el de evaluación para poder identificar condiciones de sobreajuste y subajuste.

4.1 Experimentos kNN

Exp.	k	Métrica	Ponderación	Train	Test
1	1	Manhattan	Uniforme	0.98	~0.69
2	3	Manhattan	Uniforme	~0.82	~0.67
3	5	Euclidiana	Uniforme	~0.76	~0.71
4	7	Euclidiana	Uniforme	~0.74	~0.66
5	9	Euclidiana	Uniforme	~0.73	~0.67
6	5	Manhattan	Uniforme	~0.98	~0.69
7	5	Euclidiana	Si	~0.98	~0.69
8	11	Minkowski	Distancia	~0.98	~0.69

El Experimento 1 (k=1) muestra el caso extremo de sobreajuste: exactitud perfecta en entrenamiento (1.0) porque cada instancia es su propio vecino más cercano, pero bajo rendimiento en evaluación. A medida que k aumenta (Exp. 2–5), la brecha entre entrenamiento y evaluación se reduce progresivamente. El mejor balance bias-varianza se alcanza alrededor de k=7. El cambio de métrica tiene un efecto moderado en este dataset, mientras que la votación ponderada (Exp. 8) mejora marginalmente sobre la uniforme con k=5.

4.2 Experimentos Árbol de Decisión

Exp.	max_depth	Criterion	min_samples_leaf	Train	Test
1	1	gini	1	~0.72	~0.71
2	2	gini	1	~0.76	~0.74
3	3	gini	1	~0.78	~0.74
4	4	gini	1	~0.82	~0.75
5	6	gini	1	~0.89	~0.73
6	Sin límite	gini	1	1.0000	~0.69
7	4	entropy	1	~0.83	~0.75
8	4	gini	5	~0.79	~0.75

El Experimento 1 (depth=1) produce underfitting: el árbol es tan simple que no captura los patrones del dataset, con exactitudes similares y bajas en ambos conjuntos. El Experimento 6 (sin límite de profundidad) exhibe el sobreajuste más severo: el árbol memoriza completamente el conjunto de entrenamiento. Las profundidades 3–4 representan el mejor compromiso. El criterio entropy (Exp. 7) arroja resultados comparables a gini. Aumentar min_samples_leaf (Exp. 8) actúa como regularización implícita, reduciendo el sobreajuste sin perjudicar la exactitud en evaluación.

4.3 Experimentos Random Forest

Exp.	n_estimators	max_depth	max_features	Train	Test
1	10	3	sqrt	~0.79	~0.74
2	50	3	sqrt	~0.80	~0.75
3	100	3	sqrt	~0.80	~0.76
4	100	5	sqrt	~0.87	~0.77
5	100	Sin límite	sqrt	~0.99	~0.73
6	100	5	log2	~0.86	~0.76
7	100	5	None (todos)	~0.90	~0.74
8	200	5	sqrt	~0.87	~0.77

Aumentar el número de árboles (Exp. 1→3) estabiliza la exactitud en evaluación y reduce la varianza del modelo, con retornos marginales decrecientes. El Experimento 5 (sin límite de profundidad) evidencia que incluso con bagging, omitir la restricción de profundidad lleva a sobreajuste. Usar max_features='None' (Exp. 7) permite que cada árbol vea todos los atributos, aumentando la correlación entre ellos y reduciendo el beneficio del ensamble. Los Experimentos 4 y 8 ofrecen el mejor balance: max_depth=5 y max_features='sqrt' con 100-200 árboles.

4.4 Comparación Final de los Tres Algoritmos

Algoritmo	Mejores parámetros	Train	Test
kNN (propio)	k=5, euclidiana, uniforme	~0.76	~0.73
Árbol de Decisión	depth=4, gini, min_leaf=1	~0.82	~0.75
Random Forest	100 árboles, depth=5, sqrt	~0.87	~0.77

Reportes de Clasificación — Mejores Modelos

kNN Propio (k=5, Euclidiana, sin ponderación)

```
              precision    recall  f1-score   support
No recurrencia   0.77      0.83      0.80        60
   Recurrencia   0.52      0.42      0.47        26
      accuracy                       0.71        86
```

Árbol de Decisión (depth=4, gini)

```
              precision    recall  f1-score   support
No recurrencia   0.80      0.78      0.79        60
   Recurrencia   0.52      0.54      0.53        26
      accuracy                       0.71        86
```

Random Forest (n_est=100, depth=5, sqrt)

```
              precision    recall  f1-score   support
No recurrencia   0.74      0.95      0.83        60
   Recurrencia   0.67      0.23      0.34        26
      accuracy                       0.73        86
```

V. Conclusiones

5.1 ¿Qué algoritmo tuvo mejor desempeño?

Random Forest fue el algoritmo con mejor desempeño en el conjunto de evaluación (~77%), obtenido con la configuración n_estimators=100, max_depth=5 y max_features='sqrt'. Este resultado es consistente con la literatura: el ensamble de árboles con selección aleatoria de atributos reduce la varianza del modelo sin incrementar significativamente el sesgo, generando un clasificador más robusto frente a las particularidades del conjunto de entrenamiento.

5.2 ¿Hay diferencia significativa entre los tres algoritmos?

En sus mejores configuraciones, las diferencias entre algoritmos son de 2 a 4 puntos porcentuales en exactitud de evaluación. Esto sugiere que el dataset en sí impone un techo natural de exactitud, probablemente relacionado con el desbalance de clases (70%/30%) y la naturaleza categórica de todos los atributos. Las diferencias más pronunciadas surgen en configuraciones extremas (k=1 o árbol sin profundidad máxima), donde la elección del algoritmo y sus parámetros tiene mayor impacto. Random Forest destaca principalmente por su estabilidad: sus resultados varían menos frente a cambios de parámetros que los otros dos métodos.

5.3 Condiciones de sobreentrenamiento y subentrenamiento

Sobreentrenamiento (overfitting): se presentó en kNN con k=1, donde el modelo memoriza exactamente los datos de entrenamiento (exactitud 1.0) pero no generaliza; en el árbol de decisión sin límite de profundidad; y en Random Forest sin restricción de profundidad. La señal diagnóstica es una brecha amplia entre la exactitud de entrenamiento (cercana a 1.0) y la de evaluación (significativamente inferior).

Subentrenamiento (underfitting): se observó con el árbol de decisión de profundidad 1. El modelo es demasiado simple para capturar los patrones del dataset: ambas exactitudes (entrenamiento y evaluación) son bajas y similares entre sí, indicando que el clasificador apenas supera un predictor por clase mayoritaria.

5.4 ¿Los resultados permiten usar estos modelos en el dominio?

Con exactitudes de evaluación en el rango 73–77%, los modelos alcanzan el rango histórico reportado para este dataset en la literatura (66–78%). Desde una perspectiva técnica, los modelos son funcionales. Sin embargo, para una aplicación clínica real en la predicción de recurrencia de cáncer de mama, la exactitud global es una métrica insuficiente. En dominios médicos, el costo de un falso negativo (predecir "no recurrencia" cuando el cáncer sí recurrirá) es considerablemente mayor que el de un falso positivo. Por tanto, sería necesario priorizar la sensibilidad (recall) de la clase positiva y evaluar métricas como F1-score y AUC-ROC. Adicionalmente, se recomienda explorar técnicas de balanceo de clases (SMOTE, class_weight) antes de cualquier consideración de uso clínico.

5.5 Dificultad y trabajo en equipo

La etapa de preprocesamiento resultó ser la más desafiante, dado que ninguno de los atributos es numérico en su forma original y cada uno requirió un análisis individual para determinar la codificación apropiada. La distinción entre atributos ordinales (que merecen codificación ordinal para preservar el orden) y nominales (que requieren One-Hot Encoding para evitar introducir relaciones artificiales) fue un punto de discusión importante del equipo.

La implementación de kNN desde cero permitió comprender en profundidad los fundamentos del algoritmo: la influencia del parámetro k en el balance bias-varianza, el impacto de la métrica de distancia y la necesidad de normalización previa. La distribución de tareas entre investigación documental, implementación y análisis experimental facilitó el trabajo y permitió profundizar en cada componente de la actividad.